Table 1.5 *continued*

Requirement	Comment
The system will restrict options on the information bar by processor function. When an icon is clicked, the screen represented by the icon will be displayed and the system will display appropriate participant information.	This one has lots of user interface assumptions.

1.5.4 Prototypes

The prototype wave hit software development in the mid 1980s as fourth-generation languages become popular and usable. *Prototypes* are mock-ups of the screens or windows of an application that allow users to visualize the application that isn't yet constructed. Prototypes help the users get an idea of what the system will look like, and the users can easily decide which changes are necessary without waiting until after the system is built. When this approach was introduced, the results were astounding. Improvements in communication between user groups and developers were often the result of using prototypes. Early changes to screen designs helped set the stage for fewer changes later and reduced overall costs dramatically.

However, there are issues with prototypes. Users with detail-oriented minds pay more attention to the details of the screens than to the essence of what the prototype is meant to communicate. Executives, once they see the prototype, have a hard time understanding why it will take another year or two to build a system that looks as if it is already built. And some designers feel compelled to use the patched-together prototype code in the real system because they're afraid to throw any code away.

Prototypes will always be a part of systems development. But they cannot be the one and only requirements specification. They contain too much user interface design (which can be distracting to users and designers), and they imply that more of the system is built than is actually completed. They represent only the front end of the system—the presentation. The business rules are not usually represented unless the prototype is fully functional, and this means that a lot of effort must go into the prototype. Prototypes should be used for what they are best at: user interface specification. This means that perhaps prototypes should come along a little later than the bulk of the requirements work.

1.6 Those Troublesome Requirements

The traditional tools and techniques used for gathering requirements have not served us well. We usually get ahead of ourselves and start embedding design into our requirements specifications. We spend either too little or too much effort. We create prototypes that are useful but are also distracting, and we create requirements lists that are difficult to use and don't provide any checks or balances. There must be a better way.

In my work, I focus on user goals first, and then I come up with use cases to satisfy them.

—Martin Fowler

2

Moving to Use Cases

The computer industry has struggled to find a way to represent functionality to users. We have always tended to produce what we're comfortable with: diagrams and specifications that are loaded with terminology and notation that looks a lot more like computer code than anything a user would understand. The traditional modes of expressing functionality to users early in the lifecycle are as follows:

- Requirements specifications
- Data-flow diagrams (DFDs)
- Entity-relationship diagrams (ERDs)
- Prototypes

Typically, requirements are specified in lists and expressed in terms of "the system shall." These lists are often grouped by functionality or subsystem. In such lists, it's easy to inadvertently write duplicate or conflicting requirements, and it happens often. The requirements

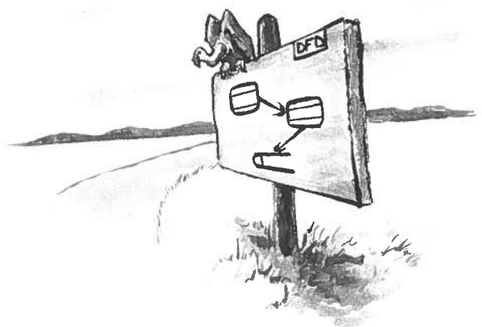


The system shall . . .

specification does not provide the users with a cohesive view of what the system will accomplish; it is merely an itemization of each of the various functions, as if the functions could be extracted and treated independently.

What requirements lists intend to provide is a comprehensive catalog of every function that the system should perform. Although it seems only natural to list these functions, it turns out that there are better ways to represent this information and provide a better structure than a list.

We recommend that requirements lists be dropped from the analyst's toolbox.



DFDs are useful for technical people but tend to confuse users.

DFDs help show a system as a set of groups of interacting processes. We consider them to represent the dynamic view of the system. The data *flows* from one process to another and then stops in a data store. External entities, such as an outside department or computer system, are referenced whenever they have involvement in the flow of data. Because DFDs focus on what happens inside the system, they are the main input into system design. In our experience, users seem to be confused by these diagrams because the line between system and user responsibility is fuzzy. DFDs also contain a lot more detail than users are prepared to wade through. Some of these details, such as how many data stores are used and what they store, do not concern the users at all.

As with requirements lists, we recommend that DFDs be dropped from the requirements analyst's toolbox. DFDs introduce many technical elements into the requirements picture that are not necessary at this point. DFDs can be replaced by use cases and class, sequence, statechart, and activity diagrams in the Unified Modeling Language (UML).

NOTE: DFDs can still be useful in design, particularly with non-object-oriented systems.

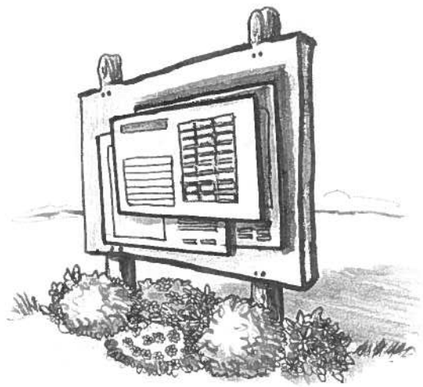


ERDs are critical to database design, but they are not meant for user consumption.

ERDs show how the data is stored in an application. They show details of entities, attributes, and relationships. This diagram can also be used to represent a logical data model and dictate the structure of a relational database. However, ERDs do nothing to show dynamic interaction, and they must always be used along with DFDs to show a complete picture to the users. The difference between a dynamic type of expression (DFDs) and a static or persistent expression (ERDs) is irrelevant and confusing to users.

We recommend that ERDs, too, be dropped from the requirements analyst's toolbox. ERDs are still required if you want to create a logical data model after requirements have been gathered, but ERDs do not provide much meaning to users and seem foreign to them.

Prototypes give users a realistic demonstration of what a system will be able to do when it is completed. These mock-up versions of the system concentrate on the user interface and omit all or most of the background coding. Prototypes are greeted with enthusiasm by users because they help them understand the possibilities much better than do paper diagrams. However, users' enthusiasm is problematic because they immediately get caught up in the details of the user interface and the mock-up data. Soon, they are spending more time requesting button moves and color changes than they spend focusing on the functionality. Prototypes also lead users to the misperception that the prototype *is* the system, and they become impatient when it takes weeks or months to develop the actual system. Prototypes also encourage the team to continue quick-and-dirty coding after actual system development begins, evolving the prototype into the final system.



Prototypes were long held to be the elixir for good requirements capture, but now we realize that they can help much more with proof-of-concept tasks.

We recommend that prototypes no longer be used as the main requirements tool. Instead, they should be used to support use cases and the business rules catalog, which become the central focus of the requirements phase.

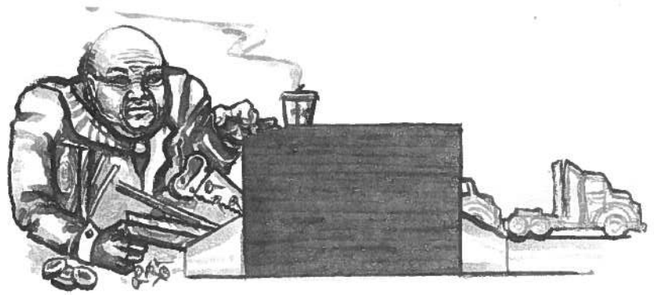
Traditional requirements specifications are often never used again after they are produced. DFDs and ERDs are useful for moving into programming and database design, but they do not seem to mean much to users. Prototypes are meaningful to users, but they encourage them to concentrate on the details of user interface implementation and not on the requirements of the system. What's needed is a new medium between users and system designers.

2.1 It's All About Interactions

Clearly, the combination of artifacts produced in our projects contains a lot of information. However, it may be that they contain too much information. What if the diagrams were to concentrate on the requirements of the system from the users' perspectives? The question is then, What do the users see?

Users view computer systems as *black boxes*. This term implies that the perspective of the application is concerned only with what goes in and what comes out. The inner workings are not important to the black-box perspective. Requirements documentation that puts everything in the context of "going in" and "coming out" has more relevance to the users who study it.

When we talk about "going in" and "coming out," we're talking about interactions. Interactions between the users and the computer system are what really matter in requirements gathering. For example, it is important that a user wants to enter numbers in timesheets and have the system print checks. But it is not important to the user that those timesheets go through 18 specific processes before the checks are produced. That's design. The users care only that the checks are produced and that they're right.



Users only care about what goes in and what comes out.

For an IT specialist, grasping this concept is very tough. We have strong tendencies to jump into the *how* before we've defined the *what*. We're typically advised not to get too detailed. But this is not quite accurate. The *what* can become very detailed. For example, the definition of printing the checks "right," in our example, could be very detailed. Information about what the system should do with calculating weekly pay based on a yearly salary, handling hourly employees, dealing with overtime, and incorporating raises at the right time can be excruciatingly detailed. However, the focus must remain on the *what* and not drift into the *how*.

Use cases are a tool that should show the *what* exclusively. DFDs, ERDs, and prototypes include *what* and *how* in their perspectives, and that can confuse users. For this reason, we recommend that use cases be the primary tool for requirements gathering.

2.2 Hello World!

Most of the programming language books we read include a "Hello World" example that shows the new language performing a simple task. Our "Hello World" use case is a *system context use case*, which is abstracted to describe a broad swath of requirements without drilling down into a lot of detail. We present it at this point to consolidate some of the ideas that we have presented about use cases. System context use cases are explained in Chapter 4.

The Origin of Use Cases

Use cases were introduced to the IT world by Ivar Jacobson and his team at Ericsson in Sweden. Their book, *Object-Oriented Software Engineering* (Jacobson et al.1992), took the computing industry by storm, and use cases have been increasing in popularity ever since. Jacobson included use cases as part of an overall system development lifecycle methodology called Objectory, which he marketed as a product and built a company around. Later, Jacobson's Objectory company and methodology were purchased by Rational Software, and Objectory, with its use cases, became part of the Rational Unified Process (Jacobson, Booch, and Rumbaugh 1999). We owe a lot to Jacobson, and this book uses many of his ideas in the definition of use cases. To his ideas we've added the practical application of use cases to solve the requirements problem. As consultants and practitioners, we have worked with use cases on the job for many years, and we understand their good and bad points. In this book, we attempt to improve on use cases, not by changing them fundamentally but by surrounding them with other artifacts and helpful hints to help create a more comprehensive requirements toolset.

A use case has two parts: the use case diagram and the use case itself. The diagram provides an overview of which interactions will be important, and the use case's text details the requirements.

Figure 2.1 is the use case diagram for a real estate system. The use case diagram shows *actors* (entities outside the program) interacting with use cases. In this case, we can see that the buyer, seller, and adviser actors interact to achieve the objective of the use case, which is to sell a property.

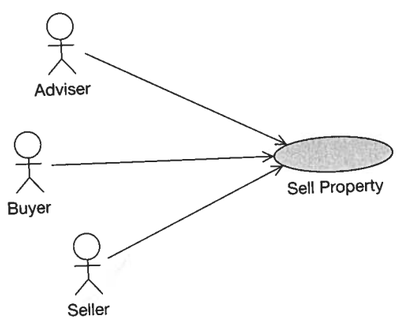


Figure 2.1 *Sell Property Use Case Diagram*

The use case diagram visualizes the system's interactions and captures the scope. The simplicity of these diagrams makes them a great communication tool, and they can be drawn using a software tool or drawn on a napkin during a scope discussion. The use case itself details what the system must do. The following use case (Use Case 2.1) is at the same level of abstraction as the diagram and supplies greater detail than the diagram.

Use Case Name:	**Sell Property**
Iteration:	**Filled**
Summary:	System Context Use Case. The seller lists the property, a buyer purchases the property, and the agent guides them through the process and offers advice, caution, and recommendations.
Basic Course of Events:	1. The seller selects an agent. 2. The system responds by assigning an agent and notifying the seller's agent. 3. The seller lists the property to sell. 4. The system responds by displaying this property in the property listing and linking it for searches. 5. The buyer selects an agent. 6. The buyer reviews the property listings by entering search criteria. 7. The system responds by displaying properties that match the buyer's search criteria. 8. The buyer finds a property and makes an offer on it. 9. The system responds by notifying the seller and the seller's agent. 10. The seller responds to the offer with a counteroffer. 11. The system responds by notifying the buyer and the buyer's agent. 12. The buyer and the seller agree to terms. 13. The system responds by recording the agreement. 14. The buyer indicates that a loan is required. 15. The system responds by locating an appropriate loan provider. 16. The buyer and the loan provider agree to loan terms. 17. The system responds by recording the terms of the loan.

Use Case 2.1 *Use Case Example (continues)*

	18. The buyer and seller close on the property. 19. The system responds by recording the details of the close.
Alternative Paths:	N/A
Extension Points:	N/A
Trigger:	N/A
Assumptions:	N/A
Preconditions:	N/A
Postconditions:	N/A
Related Business Rules:	N/A
Author:	Angela Baltimore
Date:	March 20, 2000—Facade; March 26, 2000—Filled

Use Case 2.1 *continued*

The "Basic Course of Events" section outlines the steps that actors take to achieve the use case's objective. Even highly abstracted, it speaks volumes about the system by showing you how the system will be used.

NOTE: For definitions of the four stages of iteration (Facade, Filled, Focused, and Finished), see Section 3.3.

Before continuing our discussion about use cases, let's put them in the context of the UML, of which use cases are a part.

2.3 The Unified Modeling Language



The Unified Modeling Language.

Use cases aren't a phenomenon unto themselves. They are part of a comprehensive language called the Unified Modeling Language, or UML. This language was brought into existence in January 1997 and has subsequently been adopted as a standard by the Object Management Group (OMG), an industry consortium.

The UML is a *notation*: a way to document system specifications. It is not a methodology. A notation simply tells you how to structure your system documentation. It provides the nomenclature of the diagrams and specifications that you produce. A *methodology*, by contrast, consists of a step-by-step guide to building a system. Methodologies are much more complex and contentious. The UML is not dependent on a particular methodology, nor do methodologies need to be contingent on the UML. The UML requires only that the computer system being built has object-oriented components. In fact, several of the UML diagrams, including use cases, can be used for systems that are not based on object orientation. Methodologies that work with the UML include the Rational Unified Process, the Object Modeling Technique, the Booch method, Objectory, Schlaer-Mellor, FUSION, OPEN, and many others.

Of course, the UML did not appear magically from outer space. It was the result of collaborations between three famous object-oriented methodologists: Grady Booch, James Rumbaugh, and Ivar Jacobson. The "Three Amigos," as they are called, combined their own modeling notations and also got plenty of ideas and unanimous buy-in from every other major methodologist in the industry. Through their efforts, the computer industry has a common language,[1] or notation, with which to specify object-oriented systems—something this industry has never before experienced.

SEE ALSO: http://saturn.csse.swin.edu.au/cotar/OPEN/UML98/UMLCRC.PDF

The UML is composed of nine diagrams:

- Use case diagram
- Sequence diagram
- Collaboration diagram
- Statechart diagram
- Activity diagram
- Class diagram
- Object diagram
- Component diagram
- Deployment diagram

1. There is at least one other notation that we are aware of that competes with the UML. It is called OPEN Modeling Language (OML). The OML is part of a methodology called OPEN, or Object-Oriented Process, Environment, and Notation. OPEN and OML are being championed by Donald Firesmith, Brian Henderson-Sellers, Ian Graham, and Meiler Page-Jones—an impressive roster, to say the least. OML is billed as a "superset of the UML," but it is likely to serve only as a set of recommendations to the OMG for future enhancements to the UML. We do not expect the OML to compete with the UML directly.

These diagrams, used properly within the context of a solid methodology, can convey all the necessary views of a computer system and can provide the basis for constructing, configuring, and deploying the system. There are many interdependencies between these diagrams. Construction on one type of diagram means changes to others. Software tools[2] can help manage these interdependencies. The UML lends itself well to automated analysis and design tools and also provides many possibilities for generating code from design documentation and for *reverse engineering* (creating design from code).

The UML is a well-thought-out notation that serves analysts, designers, and architects. It is comprehensive, especially when compared with earlier object-oriented notations. It is also the new standard for object-oriented systems notation, and that's the best reason of all to use it for all your documentation needs. But remember—to be successful on your project, you must choose and implement a methodology along with the UML. Adopting the UML by itself is not enough.

This book is dedicated to explaining how use cases can solve problems in requirements gathering. It is not meant to provide a tutorial for the UML, but it is important that you understand how use cases fit into the context of the UML. Books on the UML include the following:

- *The Unified Modeling Language Reference Guide* (by James Rumbaugh, Ivar Jacobson, Grady Booch), Addison Wesley, 1999
- *Applying UML and Patterns* (by Craig Larman), Prentice-Hall PTR, 1998
- *The Unified Modeling Language User Guide* (by Grady Booch, James Rumbaugh, Ivar Jacobson), Addison Wesley, 1998

2.3.1 Nine Diagrams

In the following sections we take a look at the nine UML diagrams in some detail.

2.3.1.1 Use Case Diagram

Figure 2.2 shows an example of a *use case diagram.*

Use cases, obviously, are a focus of this book. They are the driver for the rest of the diagrams. Although the UML is not a methodology and therefore does not prescribe steps, it makes the assumption that you diagram use cases before you start the other diagrams. Use cases form the basis of how the system interacts with the outside forces around it: users/actors, other systems, and other factors (date/time, special environmental conditions, and so on).

Use cases are text descriptions of the interaction between some outside actors and the computer system. Use case diagrams are graphical depictions of the relationships between actors and use cases and between a use case and another use case.

2. Our favorite analysis and design tools that support the UML are, in order of preference, Rational Rose, Advanced Software GDPro, and Select Enterprise.

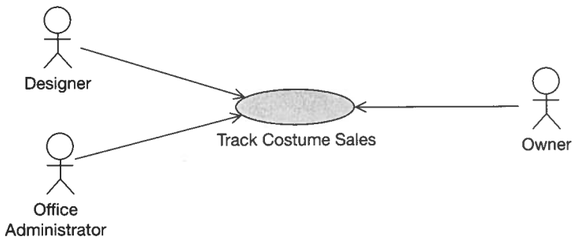


Figure 2.2 *Focused Iteration: Track Costume Sales System Context Use Case Diagram*

Use cases and use case diagrams should be documents that users can interpret easily. They are meant to be written in "user language," devoid of any "objectspeak" or implementation details. Inside each use case is a nice little package of requirements that effectively drives the rest of the system development process.

Use cases drive not only requirements gathering but also the entire software development cycle. Several methodologies, including the popular RUP, are use-case-driven. Use cases have the simplicity to represent a computer system's essence, and yet they have the power to drive the entire methodology, in the process helping to solve problems such as requirements traceability.

2.3.1.2 Sequence and Collaboration Diagrams

Sequence diagrams (Figure 2.3) and *collaboration* diagrams (Figure 2.4) show the internal workings of a use case scenario (see Section 2.4.1.4). They present a dynamic view of a system, showing how messages pass between objects to satisfy a use case.

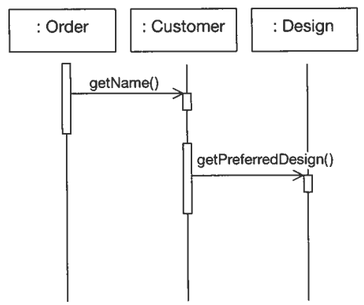


Figure 2.3 *Sequence Diagram Example*

This book uses several kinds of stereotypes to increase the relevance of use cases to requirements gathering. We discuss these stereotypes in later chapters.

Stereotypes can be introduced by anyone who uses the UML, including individual practitioners. For the purposes of this book, we use stereotyping to help clarify the concepts we are introducing. (You can read more about our new stereotypes in Chapter 3.)

In the sample use case diagram in Figure 2.11, the *Process Loan* use case is classified with the stereotype *mission-critical*. This means that all use cases marked with this stereotype must meet certain criteria; in this case, the transaction must be secure and must have less than a ten-second response time. The definition for *mission-critical* might be specified in a separate document that addresses all stereotypes.

2.4 Introducing Use Cases, Use Case Diagrams, and Scenarios

Part of our solution to the requirements-gathering problem is to deploy use cases, a deceptively simple tool that examines a computer system by expressing the interactions between the system and its environment: what goes in and what comes out. This sounds simple, but in practice it becomes very difficult. In our experience, systems professionals have an extremely hard time producing use cases that mean anything to the users. Systems people are used to dealing with systems as white boxes, not black boxes, and are focused on the *how*. Users see only the black box: the *what*.

In the book *Zen and the Art of Motorcycle Maintenance*, Robert Pirsig (1974) uses historical terms to describe differing views of a motorcycle: the *classic* view and the *romantic* view. The classic view sees a motorcycle in terms of its subsystems: power train, braking, steering, safety, and so on. The romantic view sees the motorcycle as what it can do for the rider: speed up, slow down, weave through traffic, avoid accidents, and so forth. Each view sees exactly the same motorcycle, but they see it from much different perspectives.

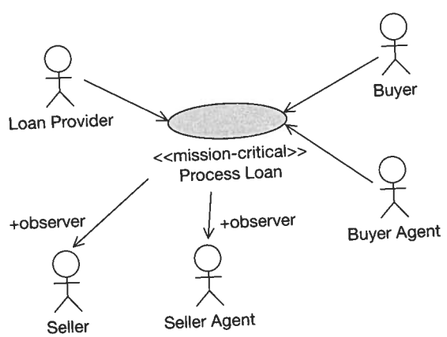
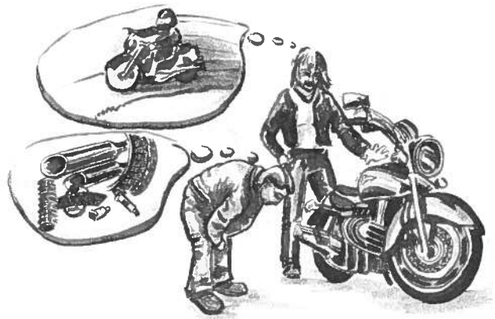


Figure 2.11 *Use Case Stereotypes*



Two people see a motorcycle in two ways: as the subsystems that make up the bike and as the things a person can do with the bike.

Use cases in the requirements activity see the system from the romantic view. They are concerned only with what the system can do for the users. This makes them extremely effective for conveying information to users because they cut out all classic viewpoints and boil down the requirements to the barest essentials.

To apply use cases effectively to the requirements-gathering process, you need to understand the goals of use cases, which are discussed in the next section. Use cases are simple in structure, but preparing them correctly is difficult. This is often the case with simple representations of complex things. In the following subsections, we explore the notation of the use case and how it fits in with the rest of the UML.

2.4.1 The Goals of Use Cases

Following are the goals of creating use cases for requirements.

2.4.1.1 Interactions That Provide Value to Actors

Use cases are meant to show the interactions between the system and the entities external *to* the system. External entities, or actors, include users, other computer systems, or external events, such as reaching a specific date or time. Each interaction should provide something of value to the external entity. If it does not, it should contribute directly to a value provided to an external entity that does not participate in the interaction. For example, a management information system may not provide value to the clerk who keys in volumes of data, but the data contributes directly to the manager, who uses the data to make business decisions. Similarly, a production accounting system may gather information from a metering system but actually provide the value to another actor, perhaps a general ledger system. When the interaction occurs between the system and an external event such as date/

time, the external event does not gain a value from the interaction; nevertheless, some other external entity should benefit from the interaction.

2.4.1.2 No Implementation-Specific Language

As mentioned previously, use cases are black-box representations, and this means that use cases do not include any *implementation-specific language*. As the use case author, you avoid using implementation-specific language by not making any assumptions about how this use case is realized in program code or the user interface. Table 2.1 provides examples of implementation-specific language and our suggestions for improving it.

Be careful about using implementation-specific terms in these areas:

- Specific people (instead of roles)
- Specific departments in the organization
- User interface widgets (buttons, menu navigation, hardware references)
- Assumptions about where the work is being done physically
- IF-THEN-ELSE statements in use case text
- Pseudocode in use case text
- The use of any kind of constraint language in use case text (such as Object Constraint Language [OCL])

Table 2.1 Removing Implementation-Specific Language

Don't	Do	Assumptions Removed
The clerk pushes the OK button.	The clerk signifies completion of the transaction.	User interface button
The account holder folds the envelope with the cash or check and deposit slip and places it inside the slot on the automated teller machine (ATM).	The account holder provides the deposit, including cash, check, and deposit summary data.	Envelope, deposit slip, slot, ATM
The customers use the mouse to click on their part selection for the zip code they specified in the pull-down box and then click on the hyperlink to the Order Finalization Web page.	The customer chooses a part from a list of available parts in the specified zip code.	Mouse clicks, pull-down box, hyperlink, Web page, navigation from one page to another

Hand in hand with the rejection of implementation-specific language, use cases should be written in the users' vocabulary. All data items and interactions should be termed and phrased using the same language that the users adopt to describe their jobs.

We call use cases that have no implementation references *context-free* use cases. It means that we can apply these use cases to any context—technical, implementation, user interface, workflow, and so forth—and they will still be perfectly applicable.

Sometimes, implementation-specific language is OK. Having stated firmly that you should never put implementation-specific details in use cases, now let's discuss when you should do it: when they are being produced or refined outside the boundaries of requirements gathering. Because use cases drive the whole lifecycle, it is reasonable to assume that they are modified throughout the lifecycle. In fact, use cases become increasingly implementation-specific as time goes on, reflecting the work being done at the time. Another term for implementation-specific, in this context, is *real*. The term *real use cases* comes from Constantine and Lockwood 1999 and is documented in Craig Larman's book (1997). Any use cases that reflect the requirements view should be saved separately, but they can also become input to use cases that help manage the work of design, construction, testing, and architecture. There is much more on this topic in Appendix A, Use Cases Beyond Requirements.

2.4.1.3 User-Appropriate Level of Detail

As you proceed through requirements gathering, use cases go from being general to being detailed. As IT professionals, we tend to migrate to the details quickly. It is important to at least start at a general level before jumping to the details. Keep in mind that use cases should always be in the users' vocabulary. This may help you keep to a suitable level of detail.

2.4.1.4 User-Appropriate Volume

At the Object-Oriented Programming System, Languages, and Applications (OOPSLA) conference in 1996, Ivar Jacobson said that a very large system should have no more than 70 to 80 use cases. Sound impossible? This means that most systems would have perhaps 20 to 50 use cases, and some small systems even fewer.

We have found that if we use good judgment, it is not only possible but also extremely wise to keep the number of use cases very small. In producing such a small number of use cases for functionality so grand, the analysts and users are forced to abstract the activities of the system until they truly represent what the system must accomplish. After the distractions and assumptions are pulled out, two processes that may seem to be unique begin to merge, and a better abstraction results.

Please note that we refer to the number of use cases and not the number of scenarios. *Scenarios* are individual instances of use cases that traverse a specific path using specific data. There can be a large number of scenarios, depending on how detailed the testing effort becomes or how much confusion exists between users and requirements analysts.

Remember that each use case represents a fairly abstract interaction between actor and system. The individual paths through the use case, shown in the Alternative Path and Exception

Path sections in the use case template (discussed in Section 2.4.3), form the basis for the detailed processing that usually requires volumes of documentation to represent.

2.4.2 How Use Case Diagrams Show Relationships

Use cases are text documents. To show two kinds of relationships—those between use cases and those between use cases and actors—we employ use case diagrams. As with all UML diagrams, use case diagrams have specific rules about notation. The following subsections explain these rules.

2.4.2.1 Actors and Roles

Use cases never initiate actions on their own. The initiator of all interactions is the actor: something outside the computer application. Actors can be people, other computer systems, or something more abstract, such as a specific date and time. Actors shown on the use case diagram should be those that interact directly with the system or are influenced directly by the system.



The use case actor.

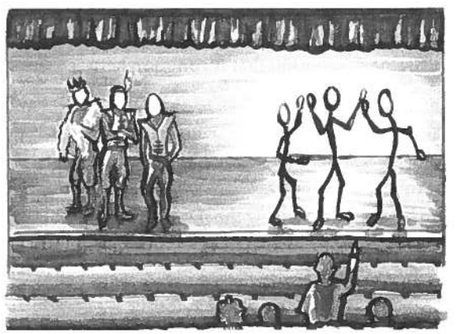
It is often hard to decide which actors to show on a use case diagram. For example, when a supplier provides an invoice to a company clerk and the clerk enters the invoice into the system, should the supplier be included in the diagram? If the supplier is shown, it is called a *secondary* actor. Our rule is that the supplier should be shown in the diagram if the supplier's behavior impacts the system in any way. For example, if a supplier sends a shipment late and that changes how the system reacts, then the supplier should be in the diagram.

Computer systems can also be actors. For example, when your system feeds postings to a general ledger system, that general ledger system becomes an actor in your use case diagram. Similarly, if other systems feed your system, they should also appear in your diagrams. A trigger of some sort can also be an actor. For example, if a system goes into action when a certain date or time is reached, such as 7:00 PM on the last workday of the month, then the date/time becomes the actor. Similarly, if a system is initiated by a condition of its environment, such as the pressure in a natural gas pipeline increasing above a certain level, then the pressure in the pipeline becomes the actor.

Actors are external to the system and outside its responsibility. It might be helpful to treat each actor as an *assumption*. You assume that the actor is out of scope and that the system will interact with it but not automate it.

Actors can be treated as use cases when analysts begin looking at the use cases to start building an object model. Actors may influence how the classes are constructed; in fact, they themselves may become classes. Therefore, it makes sense to take care in creating the right actor to interact with the use cases. (We talk about how actors in use cases influence security in Appendix A, Use Cases Beyond Requirements.)

Interactions between actors and use cases are shown with a straight, unbroken line with arrows. It is not customary to label these lines. The description of what is behind these lines is what constitutes the Basic Course of Events section in the use case text.



Good actor names are quite specific but are not linked to organizational positions or certain users.

When to Show Secondary Actors

You should show secondary actors on use case diagrams when the specific actions of the secondary actor have an effect on the responses the application provides. Another rule of thumb is to make early judgment about whether the data regarding this secondary actor will be stored in the system in any way.

Role Names

You may notice that there are names at the actor ends of the arrows in the use case diagram examples. These are *role names*. Role names are useful when the association between an actor and a use case needs information beyond the fact that they interact. In this example, these associations are special because the actors are only *observers*. This role must be defined

according to its implications. Perhaps they have read-only access to some data. This information is captured now, at requirements time, and is useful later in design.

2.4.2.2 Associations

Associations can exist between an actor and a use case, between use cases, and between actors. Let's examine each type of association.

Generalization

Generalization is a concept that is borrowed from the object-oriented world. When several use cases, for example, have something in common that can be abstracted into another, higher-level use case, they are said to be generalized.

Figure 2.12 shows two *subactors* being generalized into a *superactor*. Two types of representatives—customer and field—have behavior or attributes in common that are described under the service representative superactor. This type of generalization of actors should be done for one of two reasons.

- The superactors and subactors all have interactions with the use cases.
- There is considerable description that is common between the two subactors that would otherwise be duplicated.

The superactor should interact with use cases when all its subactors interact the same way with use cases. The subactors should interact with use cases when their individual interactions differ from that of the superactor.

Use cases can also be generalized using the same notation, but it is not used very often. We usually create one use case diagram that contains nothing but the actor generalizations.

Extend and *Include* Adornments

Use case associations stereotyped as <<*extend*>> indicate a relationship in which a special use case (the blunt side of the association) extends an original use case (the sharp side of the association). The <<*include*>> association stereotype allows use case designers to avoid duplicating steps across multiple use cases. You deploy it as a reuse strategy for use case text

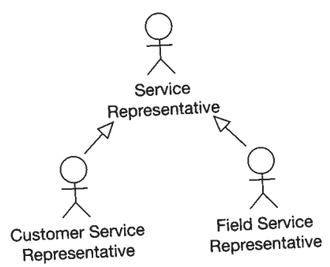


Figure 2.12 *Customer and Field Representative Generalized as Service Representative*

instead of cutting and pasting steps into multiple use case documents. These types of association, also called *adornments*, are useful when certain special cases would clutter the original use case.

In Figure 2.13, *Enter Costume Order* and *Schedule Customer Appointment* have some similar steps that can be drawn out into a separate use case called *Schedule Designer*. There may be other pieces of these use cases that are common, and they, too, could become separate use cases with *include* associations back to the owning use cases.

The *extending* use case (the blunt side of the arrow) is a special case of the *extended* use case (the sharp side of the arrow), and therefore the *extending* use case would not exist without the *extended* use case.

Be sparing in your use of the *extend* and *include* associations. It can be tempting to overengineer use cases and to go into design mode when you should really be continuing to gather requirements. Use these associations to simplify your use cases and to exploit similarity between groups of requirements. Do not use them to design a solution.

NOTE: You can think of *included* use cases the way you might think of function calls in programming. They are common pieces of logic that are reused from several calling modules.

2.4.3 The Use Case Template

Most of the UML elements are diagrams. A use case is actually a page or two of text representing each oval in the use case diagram.

You need a common template for use cases in a project. Figure 2.14 presents a sample template. Standardization on a template is more important than what the template itself looks like. You can use our template or the template supplied with the RUP. Or, best of all, you can create your own template that uniquely addresses the requirements of your organization. It is helpful, however, if the same template is used for all projects within an organization. We have these rules for creating good-quality templates.

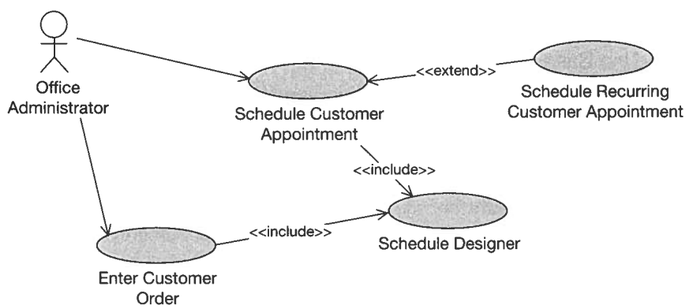


Figure 2.13 *Include and Extend Notation Example*

Use Case Name:	
Iteration:	
Summary:	
Basic Course of Events:	
Alternative Paths:	
Exception Paths:	
Extension Points:	
Triggers:	
Assumptions:	
Preconditions:	
Postconditions	
Related Business Rules:	
Author:	
Date:	

Figure 2.14 *A Sample Use Case Template*

- Make each section meaningful. For example, don't have a section for Goals and another one for Objectives. For use cases, these are probably variations of the same thing.
- Use as few sections as possible to accomplish the job. We found ourselves creating more and more boxes on our form, but if users are to use this template easily, the fewer boxes the better. Users like use case templates that allow them to speed-read when they are reviewing the documents, so organize the template with that in mind.
- Each use case will probably be a page or more of text. Give yourself plenty of space to work with and don't constrain your line length by having more than one column in a page.

2.4.3.1 Use Case Name

The use case name provides a unique identification. We prefer a unique identifier that is written in English. Long ID numbers tend to turn off users.

2.4.3.2 Iteration

Iteration relates to each of the four stages through which we see use cases progressing: facade, filled, focused, and finished (see Section 3.3). Each use case progresses through these iterations at its own pace.

2.4.3.3 Summary

In one or two sentences, describe the interaction that occurs in this use case. Try not to regurgitate the basic course of events. Remember that each section should add its own unique value. The summary section may provide some context that other sections don't contain.

2.4.3.4 Basic Course of Events

This is the meat of the use case. Describe the steps that the actors and the system go through to accomplish the goal of this use case. The actor always takes the first step, and then the system responds. This goes back and forth until the goal has been accomplished, with some value being provided to the actor.

The basic course of events represents the "simple, correct path" through the use case. This means that no errors or missteps occur, either by the actor or the system. It also means that it shows the most common path taken. For example, a retailer may find that 80 percent of all orders use purchase orders (POs), with only occasional cash, check, or credit card use. This means that the basic course of events should show the details of a PO without concern for the other types of payment.

Sometimes, courses of events are shown with separate columns for the actor's actions and the system's actions. This arrangement helps to show the interaction that occurs. However, it also makes the use case confusing if more than one actor is involved, something that may happen in some specialized cases (for details, see Section 2.4.2.1).

Although we state that use cases are text and not diagrams, we'd like to hedge that definition slightly. The structure, or template, of the use case is text. However, if some kind of picture helps the understanding of a use case, it should be included or attached to the use case. For example, it's quite acceptable to include a price schedule in table format. Furthermore, pictures of decision trees, calculations, flowcharts, or Petri nets are also quite acceptable either inside the use case or as attachments. Again, let the users' perspectives be your guide. If the users will understand the use case better with the pictures, include them.

2.4.3.5 Alternative Paths

The Alternative Paths section shows the less-common paths that need to be addressed. They include situations in which unusual types of processing occur. An alternative path would be taken when an uncommon condition occurs, such as the cash, check, or credit card payment mentioned previously.

A single use case often includes several types of alternatives. For example, the *Enter Sale* use case, which has different types of payment, may also contain alternatives relating to the type of customer, whether industrial or consumer. An industrial customer might have a standing account to which orders are collected and billed monthly, whereas walk-in consumer business is conducted with cash or credit card. Each alternative should indicate which step, in the basic course of events, is its starting point.

Alternative paths listed separately from a basic course of events allow you to avoid IF-THEN-ELSE structures. Users usually do not understand programming structures of any kind: IF-THEN-ELSE, DO-WHILE, TRY-CATCH, and so on.

Sometimes it is difficult to determine the most common alternative. It is a good rule of thumb to try to determine the statistics of use. If they are not available, choose the simplest interaction. If cash is the easiest, involving the least number of steps and verifications, then choose it as your basic course of events. The main objective is to avoid confusing the users by creating a use case that does not reflect their daily work because they seldom see that type of transaction. The Alternative Paths section is where the less common interactions are documented.

2.4.3.6 Exception Paths

Exception paths, like alternative paths, show uncommon processing. Exception paths, however, show the interactions that occur when an error happens.

For example, an exception path is taken when an actor enters an invalid date. The system provides an error message, and the actor reenters the date.

2.4.3.7 Extension Points

In Section 2.4.2.2 we talk about the *extend* relationship, an association between use cases. The *extend* relationship exists between two use cases when one use case provides an optional sequence of events that is included in the other use case. The extension points show the steps in a use case from which the extending use cases extend.

2.4.3.8 Triggers

Triggers describe the entry criteria for the use case. They are a list of the conditions that you expect to be true when an actor begins a use case. Triggers may describe a business need or be time-related, or a trigger could be the completion of another use case. Triggers answer the question "When or why will the actors enter this use case?"

2.4.3.9 Assumptions

From the project manager's point of view, this is a critical section. It is here that you document the things that you assume to be true but that might not be true. For example, if you assume that the actor has access to a current pricing list (which is not in the scope of this system) and if this use case's interactions depend on that, mention it here.

2.4.3.10 Preconditions

List the preconditions of this interaction. Preconditions are things that must be in place before the interaction can occur. They are part of the contract between this use case and the outside world.

Preconditions relate to conditions outside the scope of this use case and the computer system being developed. An example of a precondition is that a ledger exists for the incoming transaction.

2.4.3.11 Postconditions

Postconditions, like preconditions, are part of the contract between this use case and the outside world. After this use case has been completed successfully, the postconditions are satisfied.

Postconditions should be independent of the alternative path taken inside the use case. However, they do not need to be so generic that they deal with what happens when there is an error, especially an error in the input from the actor. An example of a postcondition is that a transaction is posted successfully to a ledger.

2.4.3.12 Related Business Rules

Business rules are the written and unwritten rules that dictate how a company conducts its business. This section in the use case template allows you to document or to refer to business rules that relate to the requirements presented in the use case. We find it more convenient to reference business rules than to include their entire text. In Section 3.9 we describe a business rules catalog for this purpose.

2.4.3.13 Author

You were probably expecting to see Author and Date at the top of the template, but we put them at the bottom. The most critical information should be at the top, where it can be spotted and read quickly and easily. We have heard users remark that if they can't speed-read a use case, they want it rewritten. This makes good sense, and it should be a guiding principle for designing use case templates.

2.4.3.14 Date

The Date section contains the date the use case was originally written, with references to when it was completed at each iteration. Here's an example.

Facade complete	Jan 29, 2000
Filled complete	Feb 4, 2000
Focused complete	Feb 12, 2000
Finished complete	Feb 14, 2000

These descriptions relate to how the use case template works best for us in our projects. Feel free to make the use case template your own by adding, deleting, or changing sections or changing the way it's used. We ask only that you stick to the guiding principles mentioned at the beginning of this section.

2.4.4 Paths and Scenarios

Use cases cannot tell the whole story. They are not highly detailed, and there are not many of them. To focus on the detailed interactions, we require a different tool: scenarios. Use Case 2.2 is a sample scenario that details each of the use case's interactions.

This example also demonstrates how we use the word *scenario*. In the world of requirements gathering, perhaps no term is as maligned as this one. As far as we know, there are at least three definitions of the term. For clarification, we provide each of the definitions in this section. We start with an extract of steps and alternatives from a simplified use case for a welfare application system. Then we put forth several contradicting definitions of what the scenarios would be given this use case.

Use Case Name:	**Schedule Customer Appointment**
Iteration:	**Filled**
Scenario:	1. Jenny Carmela, a fitness instructor, calls for an appointment on July 5, 2000. She needs a posing bikini and fitness outfit for the Nationals on August 22. 2. Terri, the office administrator, logs on to the system with her user ID and password and requests a list of open times when Karen, the designer, can meet with Jenny for a consultation. 3. The system responds with the following openings: • July 7 1 PM–8 PM • July 15 1 PM–9 PM • July 16 1 PM–9 PM • Etc. 4. Jenny requests July 7 at 3:30 PM, which Terri chooses in the system. 5. The system responds by recording this appointment with Karen and Jenny. 6. When Karen logs on to the system July 6 at 4 PM, the system notifies her of her upcoming appointment.
Author:	Tammie Thurber with Karen Lavton
Date:	June 6, 2000

Use Case 2.2 *Scenario of Schedule Customer Appointment*

Use Case Name: Determine Benefits Eligibility for Enrollee

Use Case Steps

1. This use case starts when the social worker enters the enrollee's name and employment situation.
2. This use case ends when the system responds with a determination of whether the enrollee is eligible for benefits and the financial extent of the benefits.

Alternative Path:

- In step 1, if the enrollee has applied for benefits previously, based on the enrollee's own disclosure, the social worker enters the enrollee's name to search for his or her previous records.[3]

Scenario Definition 1, The Alternative Path: Example of a Scenario

1. This scenario starts when the social worker determines that the enrollee has applied for benefits previously, based on the enrollee's own disclosure, and therefore enters the enrollee's name to search for his or her previous records.
2. The system responds with the enrollee's previous records, including the number of previous applications and the benefit eligibility results.
3. The social worker asks the enrollee whether he or she has applied previously for and/or received benefits through this agency. If the enrollee has applied previously for benefits, the social worker enters the enrollee's welfare recipient number or searches by name if the enrollee cannot supply the welfare recipient number.

Scenario Definition 2, The Instance

A scenario is a realization of a use case, meaning that it is one possible path (similar to scenario definition 1), but it also includes the specific data.

Example of Scenario

1. The social worker asks Edward Trueman if he has applied previously for and/or received benefits, and Mr. Trueman replies that he has applied previously.
2. The social worker provides Mr. Trueman's name as the search criterion.
3. The system provides Mr. Trueman's previous records, which state that he applied for benefits on December 9, 1997, and was determined to be ineligible on December 9, 1997, because of his current part-time employment status at Boeing Aerospace in the capacity of assembly line worker.

Scenario Definition 3, The Synonym

Scenario is a synonym for *use case* (which is a synonym for *script*). This is the way some methodologists who provided input into the UML, such as Peter Coad, identified their version of the use case.

3. Let's say that there is a government regulation that the social worker is not allowed to look for previous applications unless the enrollee discloses them.

If you choose definition 1 (the alternative path), there is no need for scenarios as separate artifacts because they merely restate what was listed more succinctly in the Alternative Paths section of the use case. If you choose definition 3 (the synonym), you do not require scenarios as separate artifacts because they *are* use cases.

Our view is that definition 2 (the instance) is the only definition that provides additional value. By including sample data, preferably taken from production files, you can prove effectively that a use case can handle that set of circumstances. Scenarios can be used in two places in the development lifecycle to prove that a use case reflects accurately what the users need it to do. First, they can be used during the requirements activity to provide immediate feedback to the users and analysts as to whether the use cases are accurate reflections of the users' needs. Second, they can be used during the testing activity to test whether the computer system reflects the requirements. This is only one way that use cases drive the rest of the lifecycle. We talk more about this in Appendix A, Use Cases Beyond Requirements.

To summarize, scenarios are *instances* of use cases (complete with production data) that effectively test one path through a use case. They are meant to demonstrate whether a use case reflects accurately the needs of the users, using terms and examples that the users understand. Scenarios are a useful vehicle for testing the validity of use cases early in the lifecycle.

The Difference between Exception Path and Alternative Path

Exception path and *alternative path* sound as if they're describing the same thing. However, they are worth differentiating. An exception path contains steps that execute if something goes wrong, such as an input from the actor that the system cannot handle or a condition or series of conditions that occurs that are not part of the functionality. An example of an exception path is an actor keying in a record for a search and the record not being found.

An alternative path is actually much closer to the basic course of events because everything happens correctly (in other words, there are no errors); an alternative path simply is less likely to happen than the basic course of events. An example of an alternative path is the use of an unusual form of payment, such as the use of a credit card when a PO is more common.

When designing the use case template for your project, you may decide to lump exception and alternative paths together under one heading or to keep them separate. In our template, we separate them because we think it helps when producing scenarios. It is more important to generate scenarios, including a wide variety of alternative paths, than to worry about exceptions.

2.5 Use Cases Apply Here

Use cases apply to a wide range of activities and not only to requirements gathering. In this section we look at the application of use cases in a couple of areas that you might not have thought of. Also, in Appendix A we show how use cases apply to other activities in the lifecycle.

2.5.1 Use Cases for Inquiry-Only Systems

Use cases make sense for any system that has interactions with the outside world. It is not outrageous to say that any computer system should interact with the outside world. The most common application of use cases occurs in transaction-processing systems. However, inquiry-only systems, such as data warehouses, are also good candidates for use case requirements gathering. For example, Use Case 2.3 is primarily an inquiry-only use case and demonstrates how a use case can capture and communicate requirements even if there are few transaction-processing requirements present.

There are a few caveats. Inquiry-only use cases are usually more abstract than transaction-processing use cases. Data warehouses are built to offer users flexible interfaces that can provide multiple views of volumes of data. This flexibility means that the users' actual interactions may not be well known at the time of requirements gathering. However, at an abstract level, the interactions are quite clear. As with transaction processing, documenting the actors for a use case for inquiry-only systems is very useful. You always need to know your audience!

Use Case Name:	**Review Listings**
Iteration:	**Finished**
Summary:	The buyer browses through the listings of properties with the help of the buyer's agent.
Basic Course of Events:	1. This use case begins when the buyer requests to view properties. 2. The system responds by providing the buyer with methods for searching keyword or specific criteria, for example: price, location, photo, size, floor plan, age, and so on. 3. The buyer's agent enters advice for the buyer into the system and also points out certain properties that the buyer may have overlooked.
Alternative Paths:	In step 3, if no agent was chosen by the buyer, this step does not exist.
Exception Paths:	None
Extension Points:	None
Trigger:	The buyer wishes to purchase a property but does not know of a suitable property yet.

Use Case 2.3 *Review Listings (continues)*

Assumptions:	None
Preconditions:	At least one property has been listed.
Postconditions:	None
Related Business Rules:	None
Author:	Ed Towson
Date:	March 23, 2000—Facade; March 26, 2000—Filled; April 1, 2000—Focused; April 7, 2000—Finished

Use Case 2.3 *continued*

2.5.2 Use Cases for Requests for Proposals

Requests for proposals (RFPs) are often used between a customer and a contractor. The customer issues an RFP to multiple contractors, the contractors bid to win the business by submitting proposals, and the customer picks a winner based on the compliance of the responses to the RFP.

RFPs can be tricky business for the customer and for the contractors. Especially with government work, the RFP for a computer system must be complete and unambiguous because the requirements that are itemized in it become the foundation for the creation of the system. Requirements lists are often used in RFPs, and because everyone is worried about legality and wants to avoid missing anything, these requirements lists can sometimes run hundreds of pages.

We suggest that you employ use cases for RFPs. If the customer were to create a set of use cases and scenarios in the RFP, the contractors could respond more easily and the solution would more likely reflect the business needs. Use cases can simplify RFPs and proposal responses in the same way that use cases simplify requirements for other types of systems requirements specifications.

2.5.3 Use Cases for Software Package Evaluation

Use cases have special applicability to software package evaluation efforts. Use cases can help clarify the "gap analysis" by comparing package functionality to business requirements. We have used this approach and have seen it used elsewhere with great success.

2.5.4 Use Cases for Non-Object-Oriented Systems

There is a predisposition in our industry to associate use cases with object-oriented systems. We believe that use cases can be used easily for non-object-oriented systems, and our experiences bear this out. Our use cases have been used to document requirements for a number of non-object-oriented systems.

The fact that use cases are part of the UML encourages the viewpoint that use cases are exclusively for object-oriented systems. However, we view use cases as a great way to document requirements for any system. In fact, we use them not only for system development efforts but also for evaluations and package implementation. Use cases are a good way to boil down the essential requirements of the required interactions, whether they are used for a new system being built or to help someone make a choice between packages that already exist.

2.6 Applying Use Cases to the Requirements Problem

We hope that you find this chapter a good primer to use cases. We love use cases, and we hope you've developed an appreciation for them as you've read through this chapter. We think they make a lot of sense as a tool for requirements, and, to this end, we've developed our own process for incorporating them into the requirements-gathering activity. Chapter 3 provides a view of our overall approach. Chapters 4 through 7, the methodology chapters, describe how to iterate through levels of fidelity with your use cases until you have everything you need to build a great system.

SEE ALSO: http://www.stsc.hill.af.mil/CrossTalk/1998/dec/ham.html